Temporal Analysis of Student Engagement in an AI Java Tutor

Quantifying Response Time Dynamics as Behavioral Signals of Engagement

Anissa Williams

DAT 507 Scientific Computing | College of Charleston | April 2026

THE PROBLEM

Why Timing Reveals Engagement

CONTEXT

This dataset captures real interactions between students and an AI-assisted Java tutoring system. Each message includes a timestamp, allowing us to treat engagement as a behavioral time series reflecting pacing, hesitation, and momentum.

RESEARCH QUESTION

How do different instructional designs shape the temporal dynamics of engagement? Do students accelerate, stabilize, or drift depending on the type of scaffolding they receive?

GAP

Most tutoring analytics focus on aggregate outcomes. Very little work examines the temporal structure of engagement — slopes, pacing rhythms, and frequency signatures — as distinct behavioral signals within AI-supported learning.

DATASET

115 sessions

2,874 total messages

1,381 user messages

3 conditions

The Three Tutoring Conditions

CONDITION 1

Character Scaffolded

AI tutor with a persona guides students through a structured 7-step instructional sequence. Scaffolding + character engagement.

41 sessions
421 messages

CONDITION 2

Non-Character Scaffolded

Same 7-step scaffolded structure, but without a character persona. Scaffolding only, no narrative framing.

39 sessions
397 messages

CONDITION 3

Direct Chat

Open-ended conversation with no structured scaffolding or character. Students interact freely with the AI.

35 sessions
545 messages

METHODS

Analytical Pipeline: Scientific Computing Tools

Goal: Extract temporal structure from session-level response time signals
Focus: how scaffolding shapes the structure of engagement — not just average outcomes.

1

Data Preparation

Cleaned logs, extracted timestamps, computed inter-message response times, filtered to completed sessions, structured each as a time series

2

Bootstrap Resampling

10,000 resamples to establish whether mean response time differences between scaffolded and direct chat are statistically robust

3

Linear Trend Estimation

OLS slope fitted per session to measure within-session acceleration or deceleration: the momentum signal

4

Moving-Average Smoothing

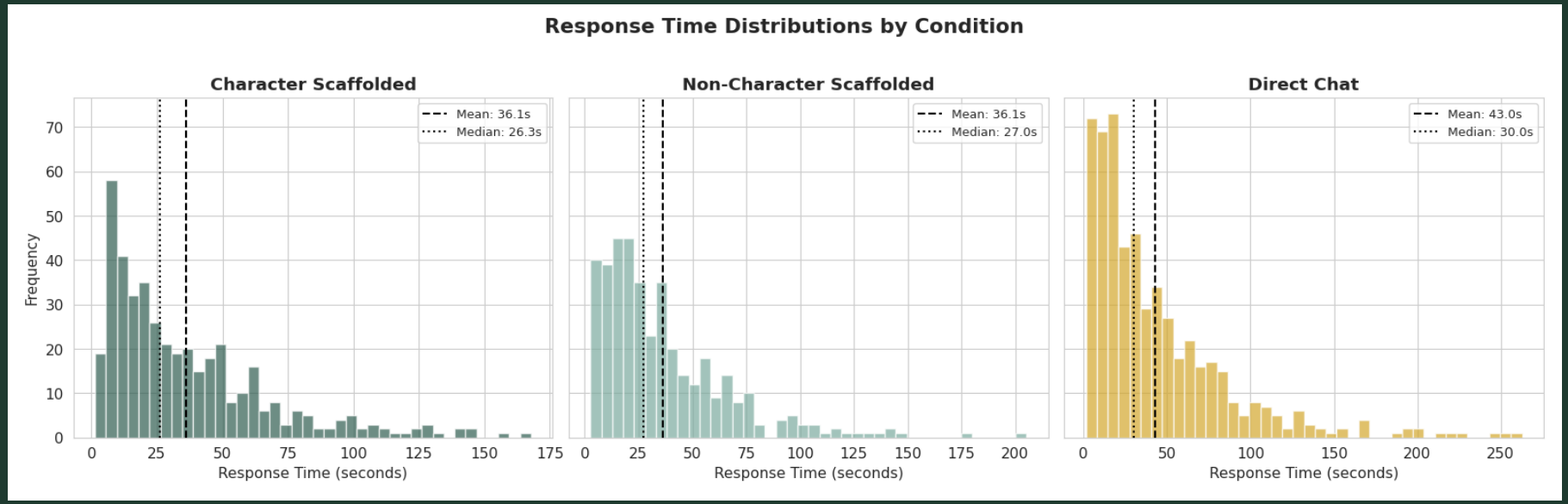
Window-3 smoothing to reveal underlying pacing structure and visually reinforce the slope analysis

5

Fast Fourier Transform

Frequency decomposition to identify rhythmic engagement signatures and distinguish multi-frequency vs. single-frequency patterns

Finding 1: Timing Separates the Conditions



CHARACTER SCAFFOLDED

36.1s mean | 26.3s median

NON-CHARACTER SCAFFOLDED

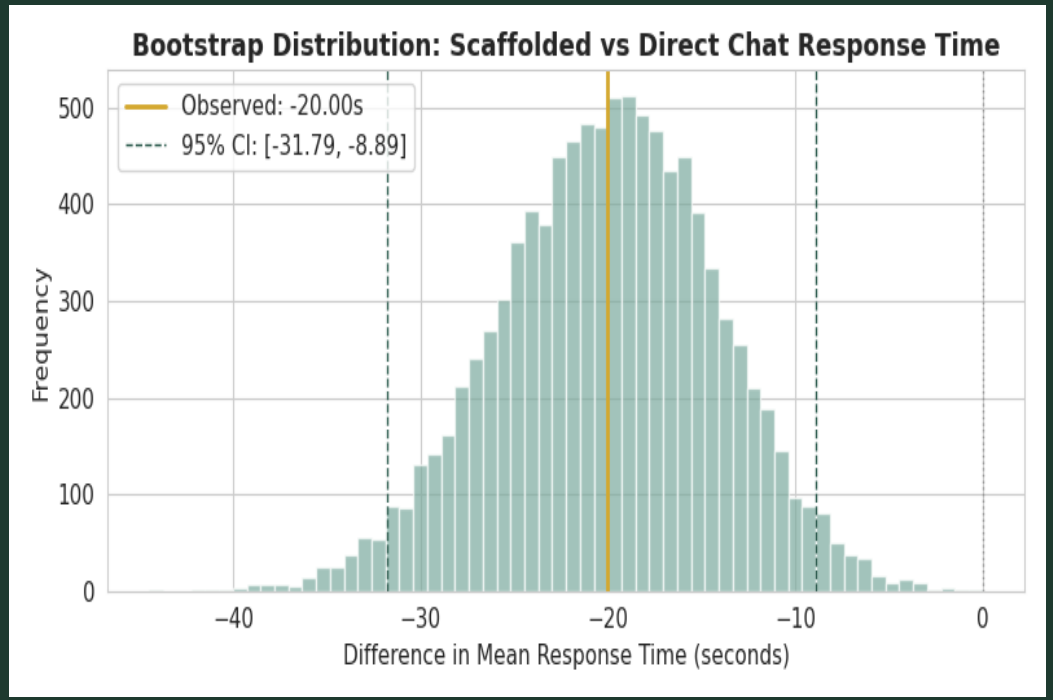
36.1s mean | 27.0s median

DIRECT CHAT

43.0s mean | 30.0s median

RESULTS

Finding 2: The Bootstrap Confirms the Pattern



OBSERVED DIFFERENCE

-20 seconds

Scaffolded – Direct Chat

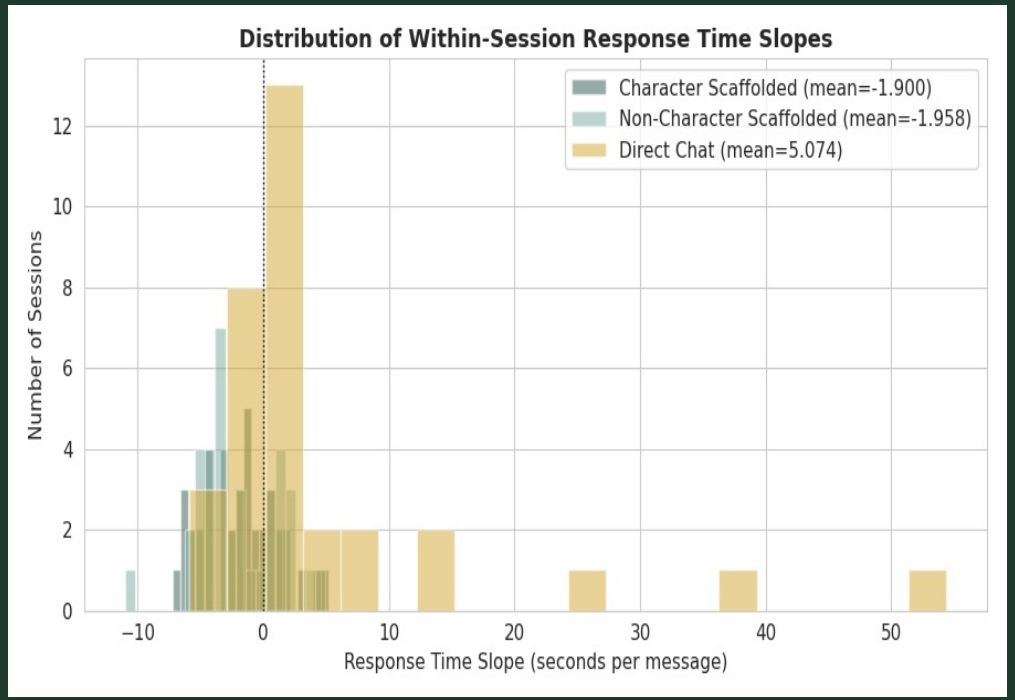
95% BOOTSTRAP CI

[-31.79, -8.89]

Entire interval is negative — CI excludes zero. Across 10,000 resamples, the mean difference stays negative, confirming scaffolded students consistently respond faster.

Scaffolding improves response efficiency in a statistically robust way — providing a solid baseline before we analyze the structure.

Finding 3: Slopes Reveal Momentum



Scaffolded: Speed Up

Character: -1.90 s/msg
Non-Character: -1.96 s/msg

Negative slopes mean students get faster as sessions progress, building momentum.

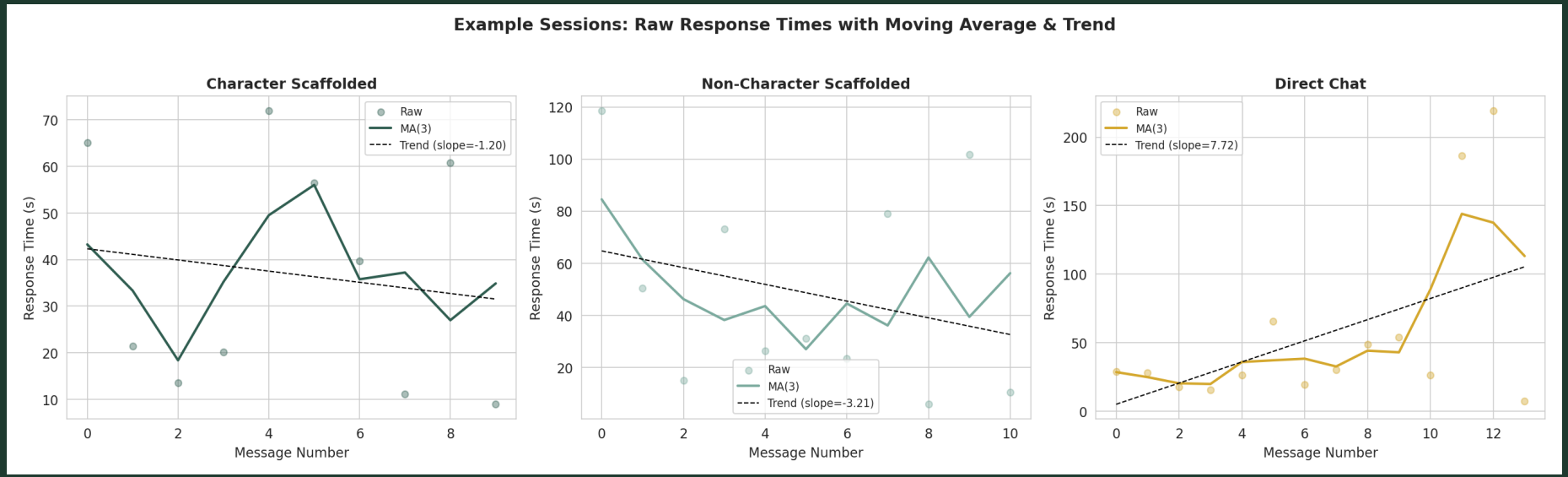
Direct Chat: Slow Down

+5.07 s/msg mean slope

Positive slopes mean students decelerate; response times grow as the unstructured session drifts.

This is the first structural sign of momentum — showing how timing trajectories differ fundamentally between conditions.

Finding 4: Smoothing Reveals Pacing Structure



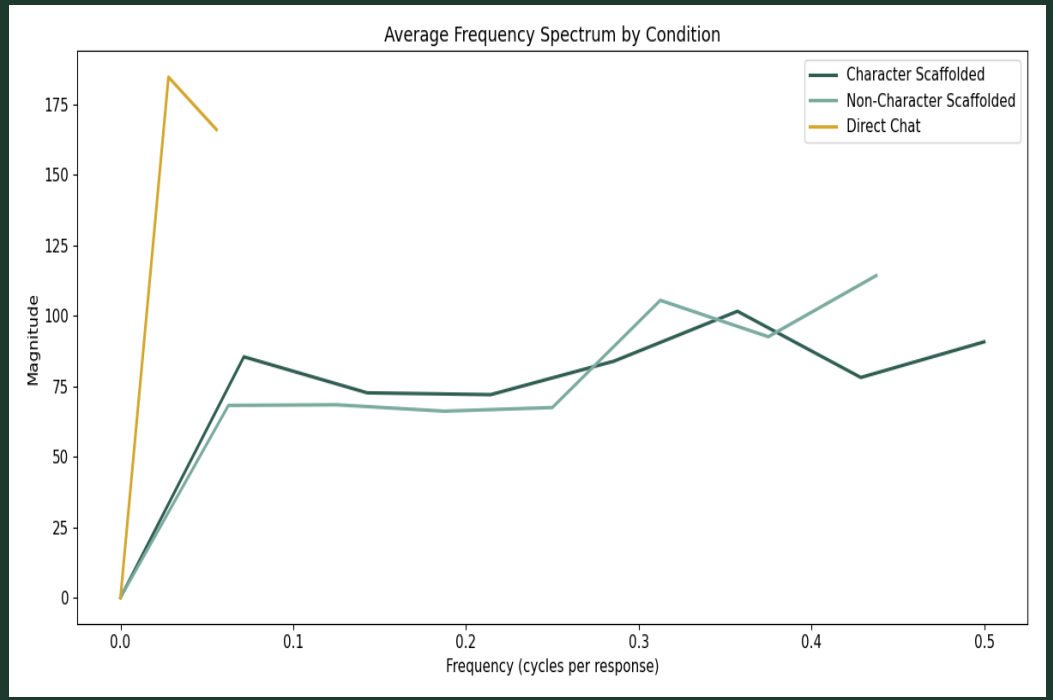
Scaffolded Sessions

Downward-trending smoothed curves reveal systematic acceleration through the instructional sequence.

Direct Chat Sessions

Upward-trending curves show increasing delays as unstructured conversation progresses without scaffolding.

Finding 5: FFT Reveals Rhythmic Engagement



DIRECT CHAT

- Dominant spike at near-zero frequency
- One large-scale trend dominates
- Single-frequency engagement signature

SCAFFOLDED CONDITIONS

- Energy distributed across frequencies
- Multi-scale rhythmic engagement
- Mirrors the 7-step instructional sequence

FFT reveals structural differences in engagement that are invisible in the time domain.

Core Computations

1 COMPUTING RESPONSE TIMES

```
# Sort messages so they are in correct chronological order
messages = messages.sort_values(
    ['user_id','session_type','message_id’])
# Compute inter-message response times w/in each
user/session
messages['response_time'] = messages.groupby(
    ['user_id','session_type'])['timestamp'].diff()
# Avoid instant replies and very long pauses
user_msgs = user_msgs[
    user_msgs['response_time'].between(1, 300)]
```

2 FITTING WITHIN-SESSION SLOPES

```
def compute_rt_slope(group):
# Provide threshold – if session too short, can’t build a
reliable slope
    if len(group) < 3:
        return np.nan
# Message index
    x = np.arange(len(group))
# Response-time series
    y = group['response_time'].values
# Fit simple linear regression (OLS) to get slope
    slope, _, _, _, _ = \
        stats.linregress(x, y)
    return slope
```

3 MOVING AVERAGE SMOOTHING

```
def moving_average(series, window=3):
# Convert input -> pandas Series & apply centered rolling
window
    return pd.Series(series).rolling(
        window=window, # size of window
        min_periods=1, # compute even at the edges
        center=True) # center the window at each point
        .mean() # take the average within each window
```

4 FFT FREQUENCY SPECTRUM

```
def compute_fft(signal):
    n = len(signal) # no. of points within signal
# Compute frequency bins for the signal
    freqs = rfftfreq(n, d=1)
# Center by subtracting the mean
    centered = signal - signal.mean()
# Compute FFT by using real-valued version (rfft)
    fft_vals = rfft(centered)
    magnitude = np.abs(fft_vals)
    return freqs, magnitude
```

Core Computations Continued

5 BOOTSTRAP RESAMPLING

```
def bootstrap_mean_diff( group_a, group_b, n_boot=10000,
seed=42):
# Set up a reproducible random number generator
    rng = np.random.default_rng(seed)
# Convert inputs -> numpy arrays
    a = np.array(group_a)
    b = np.array(group_b)
# Allocate space for the bootstrap differences
    diffs = np.empty(n_boot)
# Perform the resampling, replacing from each group
    for i in range(n_boot):
        boot_a = rng.choice(
            a, size=len(a), replace=True)
        boot_b = rng.choice(
            b, size=len(b), replace=True)
# Store the difference in means for this resample
        diffs[i] = (
            boot_a.mean() - boot_b.mean())
    return diffs
```

6 SESSION NORMALIZATION

```
def add_session_progress(df):
    df = df.copy()
# Count how many messages have occurred so far in each
user/session

    df[‘msg_order] = df.groupby([‘user_id’,
                        ‘session_type’]).cumcount()
# Total number of messages in each user/session
    df[‘msg_total’] = df.groupby([‘user_id’,
                         ‘session_type’])[‘msg_order’]
                         .transform(‘max’)
# Progress = how far student is thru session (0 to 1)
    df[‘session_progress’] = df[‘msg_order’] /
                         df[‘msg_total’].replace(0,1
    return df
```

Five Layers of Evidence

Together, scaffolding changes the structure of engagement; not just the outcome.

1	**Mean Response Time**	Scaffolded students respond ~20 seconds faster on average
2	**Bootstrap CI**	Effect is statistically robust: 95% CI excludes zero across 10,000 resamples
3	**Within-Session Slopes**	Scaffolded students accelerate (negative slopes); Direct Chat decelerates (positive slopes)
4	**Moving Average**	Smoothing reveals structural pacing differences: acceleration vs. drift
5	**FFT Spectrum**	Multi-frequency rhythmic engagement vs. single-frequency monotone drift

DISCUSSION

Challenges & Significance

CHALLENGES ENCOUNTERED

Short sessions
Sessions average ~12 messages: limits how deep the temporal modeling can go, so I used simple, interpretable methods

Uneven session lengths
Direct Chat sessions vary widely in length, so I normalized before comparing slopes and frequency structure

Skewed response times
Response times have a long tail. Bootstrap was the cleanest way to check stability without relying on assumptions about distribution.

Noisy behavioral signal
Response times jump around a lot within a session, so I used smoothing and trend extraction to detect the underlying pattern

SIGNIFICANCE

Structure > Aggregates
Time-series methods reveal engagement patterns invisible to mean-based comparisons

Scaffolding shapes dynamics
Not just faster responses: scaffolded students build momentum and exhibit multi-frequency rhythmic engagement

FFT as a novel lens
Frequency analysis of tutoring interactions is uncommon in educational research; this opens new analytical directions

Response times as signal
Demonstrates that timestamps alone, analyzed as time series, contain rich behavioral information about learning engagement

Key Takeaways

01

Scaffolding Produces Structured Engagement

Scaffolded sessions show accelerating, multi-frequency engagement patterns; students build momentum as they move through the instructional sequence.

02

Direct Chat Produces Drift

Without scaffolding, sessions show decelerating, single-frequency engagements; students slow down and lose rhythmic structure over time.

03

Time-Series Methods Reveal Hidden Patterns

Slopes, smoothing, and FFT expose engagement dynamics that are completely invisible to aggregate statistics like means or medians.

04

Response Times Are a Meaningful Signal

Timestamps alone, analyzed as time series, contain rich behavioral information about learning engagement in AI tutoring systems.

Future directions: wavelet analysis for time-localized frequency changes, longer sessions for richtr temporal modeling, and linking engagement dynamics to content difficulty.

Thank You

Questions?

Anissa Williams

DAT 507 Scientific Computing | College of Charleston

April 2026